HLD – High-Level Design

1. System Overview

Problem definition
A customer support assistant for a Samsung mobile user needs to:

- Quickly answer questions from a **product manual PDF**.
- Be able to **escalate complex or sensitive queries** to a human agent.
- Have **clear, traceable logic** for routing and decisions.

Scope

- The system ingests a **Samsung mobile user manual PDF**.
- Stores it in a **vector database** for retrieval.
- Uses a **LangGraph-based workflow** to:
    - Retrieve relevant chunks.
    - Generate answers (in demo: hardcoded; in production: LLM-based).
    - Route queries to either auto-answer or **HITL (Human-in-the-Loop)**.
- The interface is a **CLI** for this version, but can be extended to web or API.

---

2. Architecture Diagram (Text Description)

- **User interface**
    - CLI → rag_graph.py
- **Document ingestion pipeline**
    - ingest.py
        - PyPDFLoader("manual.pdf") → RecursiveCharacterTextSplitter → dummy_embedding → Chroma.
- **Embedding system**
    - dummy_embedding (384-dim dummy hash-based embeddings).
- **Vector database**
    - Chroma collection support_kb (persisted at ./chroma_db).
- **Retrieval layer**
    - db.as_retriever(k=4) retrieves top-4 chunks by similarity.
- **LLM processing layer**
    - In this demo: **hardcoded Samsung-like answers** in process_node.
    - In production: pluggable LLM (e.g., OpenAI, Gemini, Hugging Face).
- **Workflow orchestration (LangGraph)**
    - StateGraph(GraphState) with:

    - process node: retrieval + answer logic + routing.
    - output node: applies HITL logic and formats final answer.
- **HITL system**
    - hitl_queue: List[HitlEntry] stores escalated queries + initial answers.
    - Human agent can inspect hitl_queue later and set human_response.

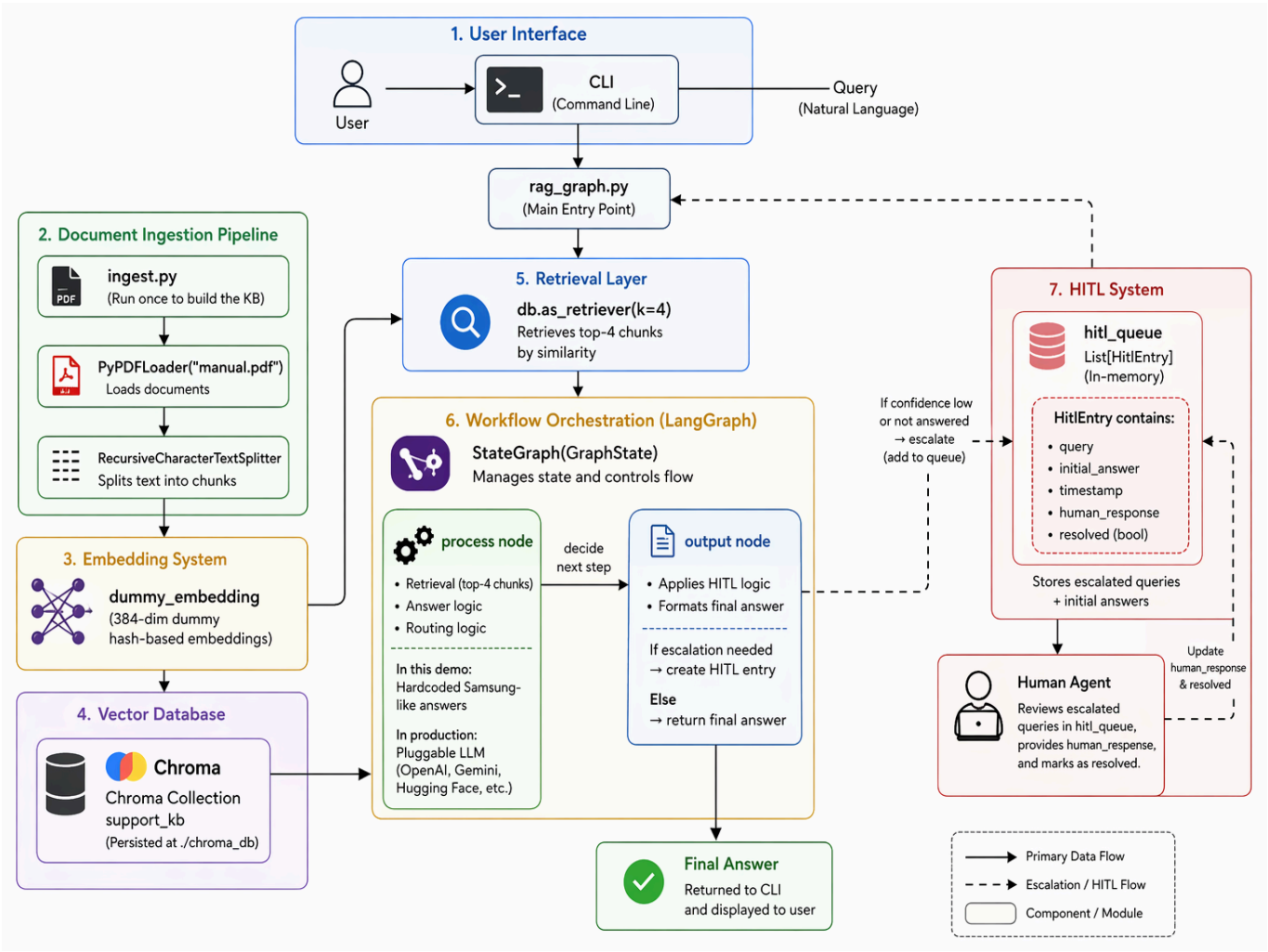

Figure 1: Architecture Diagram

3. Component Description

- **Document Loader**
  - PyPDFLoader: reads manual.pdf into Document objects.
- **Chunking Strategy**
  - RecursiveCharacterTextSplitter(chunk_size=512, chunk_overlap=64) to keep context while limiting chunk size.
- **Embedding Model**
  - dummy_embedding: MD5-based fake 384-dim vector (for demo only).
- **Vector Store**
  - Chroma: lightweight, persistent vector DB; easy to run locally.
- **Retriever**
  - db.as_retriever(search_kwargs={"k": 4}).
- **LLM**
  - Currently **hardcoded responses** in process_node for Samsung-related queries (e.g., model number, features, warranty, refund).
  - Designed to be replaced by real LLM in production.
- **Graph Workflow Engine**
  - LangGraph.StateGraph(GraphState) orchestrates process → output.
- **Routing Layer**
  - In process_node:
    - Low-confidence / no-chunk queries → may restrict escalation.
    - Sensitive-word presence → needs_hitl = True.
- **HITL Module**
  - HitlEntry class stores query, chunks, initial_answer.
  - hitl_queue accumulates escalated entries for later human review.

---

4. Data Flow

- **From PDF to vector DB**
  - manual.pdf → ingest.py → load → split → embed → Chroma.
- **Query lifecycle**
  - python rag_graph.py
  - User types query → ask(query) → graph.invoke({"query": query})
  - process_node:
    - retriever.invoke(query) → chunks.
    - If chunks or query matches predefined Samsung patterns → return **hardcoded answer**.
    - Else → generic demo message.

- output_node:
    - If needs_hitl → store in hitl_queue and return escalation message.
    - Else → return generated answer.

---

5. Technology Choices

- **Why ChromaDB**
    - Simple, in-memory / file-based; ideal for local demo and PoC.
    - Minimal setup, no extra server, easy persistency.
- **Why LangGraph**
    - Lets you build **stateful workflows** with clear nodes and edges.
    - Easy to add new nodes later (e.g., validation, summarization, feedback-loop).
- **LLM choice**
    - In this demo: **no real LLM**; answers are hardcoded for Samsung-manual topics.
    - In production: any LLM (e.g., Gemini, OpenAI, Hugging Face) can be plugged into process_node.
- **Other tools**
    - langchain_community.document_loaders.PyPDFLoader
    - langchain.text_splitter.RecursiveCharacterTextSplitter
    - langgraph.graph.StateGraph

---

6. Scalability Considerations

- **Handling large documents**
    - Chunking strategy already splits PDF into manageable pieces.
    - In production, embeddings become real (e.g., sentence-transformers / OpenAI), which scales better.
- **Increasing query load**
    - The current code is synchronous CLI; in production it can be wrapped in an **async web server** (e.g., FastAPI).
- **Latency concerns**
    - Dummy embedding is fast but not meaningful.
    - Real embeddings + LLM will add latency but stay acceptable for support-bot use-case if proper batching and caching are added.